Thakaa at AraGenre 2026: Definition-Guided LLM Judging with Full-Taxonomy Agreement-Gated Correction for Hierarchical Arabic Genre Classification

Hassan Alqaeri **Meshal Alamr** **Abdullah Aldahlawi**
Thakaa Advanced Company for Information Technology, Riyadh, Saudi Arabia
{h.alqaeril, m.alamr, aldahlawi}@thakaait.net

Abstract

We report the first-place system in AraGenre 2026 (0.7352 Hierarchical Macro F1, 18 teams). The task assigns each of 27,972 hidden-test Arabic segments one of 6 broad and one of 74 specific genres, given only as English prose definitions, with the specific-label inventory *disjoint* from training. Our pipeline updates no model parameters but is not free of task-specific effort: the judge prompt carries 74 hand-written cues, and components were selected on the public leaderboard. A family-restricted, definition-guided LLM judge scores candidate genres setwise under a hierarchically constrained decoding scheme, reaching 0.7139. That base led on Specific Macro F1 but trailed on Broad Macro F1, so we add a correction stage: two instruction models re-predict the specific genre, each shown the *entire* 74-definition taxonomy, and a label is edited only where both agree against the base. This lifts Broad Macro F1 to 0.892, largely by suppressing a *topic trap* in which the subject a text discusses displaces the genre it instantiates. In a post-evaluation audit of instances the base labelled book descriptions, Religious calls fall $394 \rightarrow 233$ once the prompt states the type-versus-topic convention, and to 119 once it also shows the full taxonomy. Refinements and an attractor drain reach 0.7352.

1 Introduction

Genre is what a text *does* rather than what it is about: the same subject appears in a contract, a news report and a textbook, and only the first is useful to someone assembling a legal corpus. Sorting documents this way is what makes an archive filterable and a corpus assemblable by register, and Arabic has far less of this work behind it than English. AraGenre 2026 (El-Haj et al., 2026) frames the problem as one of generalisation: systems receive English genre definitions and limited synthetic training data, but the hidden benchmark contains noisier naturally occurring Arabic spanning Modern Standard Arabic, Classical Arabic, and multiple dialects, with *previously unseen genre–definition combinations*. Because the 74 specific test genres are disjoint from training, memorisation is useless and semantic understanding of communicative function is required. We placed 1st of 18 teams (Table 2) on the official ranking metric, Hierarchical Macro F1: the mean of Broad and Specific Macro F1.

Our contributions are:

1. A hierarchically constrained, definition-guided LLM judge that reaches 0.7139 Hierarchical Macro F1 with no parameter updates.
2. An account of why one predictor fails the two tiers differently: broad accuracy is the family-accuracy of the specific prediction, whereas the gap between it and Broad Macro F1 is consistent with uneven performance across broad classes, so correction must be tier-specific (§4).
3. An agreement-gated correction stage in which each verifier sees the full taxonomy, with a three-arm ablation splitting the topic trap between an explicit type-vs-topic instruction and a visible label to move the error onto.
4. An attractor-drain step targeting the macro-F1 failure mode of heavily predicting a few generic classes; its only gold-scored estimate is on dev, where it was also tuned.
5. Empirical validation on the hidden test: 0.7352 Hierarchical Macro F1, ranked 1st of 18 teams, with the major system configurations scored as their own submissions.

The main challenges were a specific-label inventory disjoint from training, the topic trap of §4, and classes with no written signal (song dialects, textbook levels).

2 Related Work

Arabic and hierarchy-aware HTC. Arabic pretrained models (Antoun et al., 2020, 2021) underpin most classification work on the language, including hierarchical variants (Bouchiha et al., 2025, 2026), and a large literature models the label hierarchy explicitly (Zhu et al., 2023; Cai et al., 2024; Zeng et al., 2025; Wang et al., 2025; du Toit and Dunaiski, 2025; Zangari et al., 2024). These closed-label supervised setups do not transfer to AraGenre's unseen, definition-specified labels; we instead enforce the hierarchy structurally, forcing broad = parent(specific).

Zero-shot HTC, LLM judging, and consensus. Prior systems reach an unseen label space through prompt ensembles (Xia et al., 2025), LLM-built prototypes (Zhang et al., 2025) or rewritten taxonomies (Golde et al., 2026); we leave the definitions unrewritten and vary how much of the taxonomy is visible, though the base judge appends hand-written cues to each (§4). Our two-stage design retrieves candidate definitions (Chen et al., 2024) and compares them with an LLM (Sun et al., 2023; Zheng et al., 2023), with Qwen3-Embedding (Qwen Team, 2025) bridging Arabic and English; the chain-of-thought pass follows reasoning-based HTC (Jiang et al., 2025; Pan and Wu, 2025). Ensembles of diverse predictors (Dietterich, 2000) appear in LLM form as juries, blending, mixtures of agents, debate and self-consistency (Verga et al., 2024; Jiang et al., 2023; Wang et al., 2024; Du et al., 2024; Wang et al., 2023), which aggregate free-form text or resample one model; we instead require two models to emit the *same* label.

3 Background

Each instance is `{id, text}`; systems output one `broad_genre` (of 6: Informative, Creative, Interactive, Learning, Legal, Religious) and one `specific_genre` (of 74), using exact taxonomy label names, each with an English definition. The released data are tiny and synthetic-like (train: 140 items / 7 specific genres; dev: 110 / 6), and the specific-label inventories are *100% disjoint* across train, dev and test, so no supervised label set transfers. The *hidden test* has 27,972 naturally occurring instances over 74 specific genres in 6 families: Informative (42), Learning (12), Creative (7), Religious (6), Interactive (4) and Legal (3);

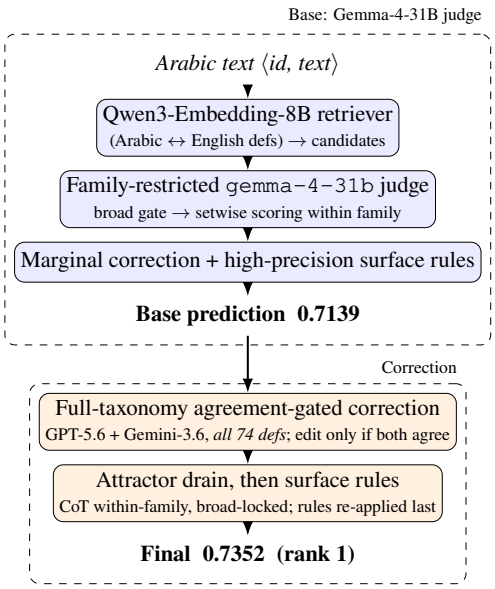

Figure 1: System overview. The base judge yields 0.7139. An agreement-gated correction, in which each verifier sees the *entire* 74-definition taxonomy, is followed by a within-family attractor drain, reaching 0.7352 (rank 1).

Informative is dominated by 20 job domains and 13 book-description topics, and Creative holds 5 song dialects. Missing predictions are counted incorrect, and Informative dominates our predictions (∼50%).

4 System

4.1 Base: family-restricted definition-guided judge

The base system turns the 74-way choice into a within-family one (Figure 1): retrieval proposes candidates and a judge scores only siblings of a gated broad genre. The setwise judge is `google/gemma-4-31B-it` (Gemma Team, Google DeepMind, 2026), served through vLLM under the local alias `gemma-4-31b`; API model strings are pinned in Appendix H. Each text passes through four steps. (i) `Qwen3-Embedding-8B` (Qwen Team, 2025) encodes the Arabic text and the English definitions, and proposes candidate specific genres by cosine similarity. (ii) Decoding is hierarchically constrained, all siblings for small families and top-k retrieved for large topical ones, so that broad = parent(specific) and broad *accuracy* equals the family-accuracy of the specific prediction. Broad Macro F1 does not: it averages per-class F1, and the gap between the two is what

the diagnosis below exploits. (iii) The judge scores each candidate 0–100 against its definition in one setwise call (Sun et al., 2023), and we take the argmax after a light marginal correction (Appendix G). (iv) A few high-precision surface rules override the judge only on near-unambiguous markers (Qur'anic mushaf orthography → `quran`; isnād openings → `hadith`; hand-screened samples showed nothing visibly violating the intended rule semantics, which is not a precision estimate; emoji/hashtag short posts → Interactive subtypes). Appended to each definition the judge also sees a contrastive discriminator, one per test genre: a single Arabic line naming the signal that separates it from its family siblings, written from the released English definitions and Arabic linguistic knowledge without reference to labelled data (Appendix H). On the hidden test this base pipeline scores 0.7139.

Diagnosis from the scoreboard. The leaderboard exposed the structure: against the system then ahead of us, our base led on Specific Macro F1 (0.5487 vs. 0.5339) but trailed on Broad Macro F1 (0.8792 vs. 0.90), so the whole deficit sat in the broad tier. Broad accuracy (0.9003) exceeded broad macro, indicating uneven per-class performance; reading our own predictions showed cross-family leakage, with Bible passages labelled poetry or encyclopedia entries and news labelled legal.

4.2 Correction: full-taxonomy agreement-gated re-prediction

We re-predict the specific genre with two frontier instruction models (GPT-5.6 Luna (OpenAI, 2026) and Gemini-3.6 Flash (Google, 2026)), *feeding each the entire 74-definition taxonomy* (≈2.9K tokens) grouped by broad genre, and asking for the single best specific label. The two-model design is motivated by the ensemble argument that combining diverse predictors beats any single one (Dietterich, 2000), in its LLM form (Verga et al., 2024; Wang et al., 2024), but we never submitted either verifier alone, so agreement here is a conservatism device and not evidence of an ensemble effect. (Claude Sonnet 5 (Anthropic, 2026) was run on the diagnostic pool but excluded from the deployed correction on cost grounds.)

What suppresses the topic trap. Given only the 6 broad definitions, all three verification models classified by subject matter rather than by communicative function: a blurb summarising an Islamic work *performs* the function of a book description, so the taxonomy files it under Informative while every broad-only verifier answers Religious. The full-taxonomy prompt also carries an explicit type-vs-topic instruction, so a post-evaluation three-arm ablation over the same 1,403 instances separates the two (one model, GPT-5.6, identical pool): 394 are called Religious with neither, 233 with the instruction alone (-41%) and 119 with the full taxonomy (-49%). The two are not interchangeable: the instruction lowers Religious calls on the book descriptions but raises them on the rest of the pool, whereas the full taxonomy lowers them on both while raising them on base-Religious instances, widening the separation from 21 to 41 percentage points (Table 3). Base-family agreement rises in five of the six families (Appendix E); Appendix C gives a worked example.

Conservative application. The first model runs over all 27,972 instances, the second only over the 8,041 where the first disagrees with the base, and a correction is applied only where both name the same label. Of the 1,178 proposed cross-family moves, 460 fall in seven author-screened directions and are applied; the other 718, such as Informative→Learning, are rejected (Appendix I). Within-family refinements are applied wherever both models agree.

4.3 Attractor drain: redistributing to low-frequency siblings

Specific Macro F1 averages F1 over all 74 classes, so a class that is systematically under-predicted has low recall and low class-level F1. This stage is partly transductive: reading our own hidden-test predictions during the evaluation phase revealed five *attractor* classes broad enough to absorb any text the judge cannot place, leaving their rarer siblings almost unused, so redistributing labels from an attractor to those siblings moves the mean far more than accuracy. The list comes from that prediction distribution, not from held-out gold. The drain is deterministic: when the base label is an attractor class and a separate chain-of-thought re-judging pass (Wei et al., 2022) names a different sibling in the same family, the sibling replaces it (broad never changes). The surface rules are re-applied afterwards as a guard (Appendix I). On the base alone it moves 554 instances (Appendix F); in the final pipeline the correction runs first, so it fires on 383.

5 Experimental Setup

Data splits. We update no model parameters, and each configuration predicts automatically once fixed, but the configurations were adapted between submissions from leaderboard feedback and from reading test inputs and our own outputs. An input is `{"id","text"}` with Arabic text such as مطلوب سائق باص 24 راكب في شركة في دبي جبل علي (test id 24486, *matlūb sā'iq bās 24 rākib fī sharika fī Dubayy Jabal 'Alī*, "bus driver wanted, 24-seater, for a company in Dubai, Jebel Ali"); our submitted system pairs it with `broad_genre` = Informative and `specific_genre` = `drivers_and_delivery_jobs`.

Preprocessing. We apply no text normalisation; retriever inputs are capped at 512 tokens and verifier inputs at 1,500 characters. Normalisation erases two orthographic signals: heavy diacritisation marks 77% of the texts we label `quran` and 43% of poetry against 1% elsewhere, and emoji mark 28% of Interactive texts against under 1% elsewhere (Appendix I).

Tools and metrics. Qwen3-Embedding-8B under `sentence-transformers`, vLLM for the local judge, vendor SDKs for the verifiers; we report Macro F1, Weighted F1 and Accuracy at both tiers (Appendix H).

Cost. The correction makes 36,013 calls at a ≈3.0K-token prompt each, about US$50; the base needs no paid API (Appendix H).

6 Results

6.1 Development-set results and component transfer

Re-running the pipeline on dev (110 examples, official gold) separates what transfers from what does not. No class list is carried over, and the two dev classes the drain fires on are the two most-predicted there, the same two under both judges, so no gold and no particular base is needed to find them. On dev it moves five labels for $+0.042$ (95% CI $[+0.010, +0.082]$), but four are a single confusion and the component was tuned on these same items, so this is descriptive, not held-out, evidence. The correction stage does not: dev has no `*_book_description` and no scripture, so the trap it repairs has zero instances, and both corrections that fire are wrong. Residual dev disagreements stem from a convention clash between splits (Appendix B).

System	Split	Hier. F1
Official baseline (embedding sim.)	dev	0.4658
Our judge (Gemma-4-31B)	dev	0.8794
Our judge (GPT-5.6)	dev	0.8943
Gemma-4-31B + correction	dev	0.8557
GPT-5.6 judge + attractor drain†	dev	0.9358
Genre-general judge, no retrieval	test	0.6067
+ embedding retrieval gate	test	0.6812
+ family-restricted judging	test	0.6882
+ judge prompt, correction, rules	test	**0.7139**
+ agreement-gated correction (subset)	test	0.7224
+ cross-family moves, all instances	test	0.7256
+ within-family refinements, drain	test	**0.7352**

Table 1: Main progression on both splits. Dev is near-degenerate (its 6 specific genres map one-to-one onto 6 broad, so all three F1 metrics coincide), so the hidden test is our primary evaluation. *Subset* corrects only instances whose broad label the base judged least secure. Rows are successive submissions, not single-component ablations. †applied to the GPT-5.6 judge, tuned on dev, so not held out.

System	Hier	SpM	SpW	SpA	BrM	BrW	BrA
Base (ours)	.714	.549	.584	.591	.879	.900	.900
+ correction	.722	.553	.592	.598	.892	.910	.910
Final (rank 1)	**.735**	**.573**	**.613**	**.619**	**.898**	**.914**	**.914**

Table 2: Official metrics for our three scored systems on the hidden test, rounded to three decimals (Sp/Br = Specific/Broad; M/W/A = Macro F1, Weighted F1, Accuracy). Correcting the verification subset lifts the broad tier by $+.013$ Broad Macro F1; the final row folds in the cross-family moves, the within-family refinements and the drain, together worth $+.020$ Specific Macro F1.

7 Analysis

Broad-only prompting put 157 book descriptions under Religious unanimously across all three models, which the full taxonomy largely reversed; one model alone disagreed with the base on ∼29% of specifics. Every alternative correction we tried fell below the base (Appendix D).

Qualitative prediction audit. Three patterns dominate: cross-family topic leaks, heavy attractor prediction, and song-dialect classes where 87–99% of texts we assign to them carry no marker from our cue inventory (App. C).

8 Conclusion

A definition-guided judge, an agreement-gated correction and an attractor drain placed 1st of 18.

Limitations

What is and is not a score. Every hidden-test *score* we report is an official evaluation-phase submission (Appendix I); we made none after the deadline. The instruction-only ablation arm (§4) was run after the phase closed, and the descriptive tables in Appendices C, E, F and I are local analyses of submitted outputs, not scored runs. **Validation.** The hidden test has no released gold and dev is near-degenerate, so components were selected on the public leaderboard and by manual reading; the reported progression therefore doubles as the selection signal, which risks fitting the test distribution. No hidden-test gain carries a confidence interval, so those gains are leaderboard-measured rather than statistically certified; Appendix I gives the scale of a class-level effect. The one component testable against gold is the drain, where a paired bootstrap on dev puts its gain above zero (§6.1). **Unmeasured components.** We never ran the judge with and without the 74 discriminators and per-family cue lists, so we cannot separate their contribution from the rest of the prompt. **Partly transductive.** The marginal correction subtracts a per-genre mean computed over the whole test set (Appendix G), so the base cannot label a document in isolation, and two further choices were derived by reading our own predictions over the blind inputs: the five attractor classes (§4) and the seven gated broad-move directions (Appendix I). No hidden label was inspected and no prediction was edited by hand, but the method is not fully inductive and neither choice is validated on gold. **An untested comparison.** We ran GPT-5.6 over the full test set but never submitted it as a system, so the claim that a cheap base judge with conservative correction beats direct full-taxonomy prediction is untested. **Cost and ceilings.** Correction cost scales with test size, and our cue inventory has low coverage among texts we predict as song dialects, so we lack reliable surface cues for those distinctions.

Ethics Statement

This work uses only the publicly released AraGenre data and collects nothing new. In line with the shared-task rules all predictions are produced automatically: no submitted label was hand-edited and we did not reverse-engineer the hidden evaluation labels. We did read system outputs and their texts to design the automatic rules, but that is inspection of our own outputs, not annotation of the evaluation set, and no such judgement enters a submission except through a deterministic rule.

The taxonomy spans religiously and culturally sensitive categories, which we treat descriptively as text-type labels: a misclassification carries no judgement on the authenticity, orthodoxy or quality of a text, and models trained on skewed corpora may under-serve lower-resourced dialects. Two misuses would concern us. Dialect is not provenance: pointed at ordinary prose, the classifier reads as a guess at where a writer is from, and we have no evidence it supports that use. And because the labels separate religious from informative text, genre output could be used to route or suppress material by register rather than by content. The system assigns text types, it does not assess them, and a 0.57 Specific Macro F1 is not a basis for an automated decision about a document or its author without a person in the loop.

Our correction stage sends Arabic text to third-party hosted APIs; the inputs are already-public benchmark data, but the same pipeline should not be applied to private text without review. Reliance on hosted models whose cost and behaviour may change limits equitable reproducibility; the base retriever-plus-open-judge pipeline is the self-hostable alternative.

Acknowledgments

We thank the AraGenre 2026 organisers for designing and running the shared task, for the released taxonomy and definitions, and for their prompt support throughout the evaluation phase. We also thank the maintainers of the open models and libraries our pipeline builds on.

References

Ilias Aarab. 2026. BTZSC: A benchmark for zero-shot text classification across cross-encoders, embedding models, rerankers and LLMs. *arXiv preprint arXiv:2603.11991*.

Muhammad Abdul-Mageed, Chiyu Zhang, Abdel-Rahim Elmadany, Houda Bouamor, and Nizar Habash. 2021. NADI 2021: The second nuanced Arabic dialect identification shared task. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop*.

Anthropic. 2026. Claude Sonnet 5. Model documentation, https://www.anthropic.co

m/news/claude-sonnet-5. Accessed 23 August 2026.

Wissam Antoun, Fady Baly, and Hazem Hajj. 2020. AraBERT: Transformer-based model for Arabic language understanding. In *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools*.

Wissam Antoun, Fady Baly, and Hazem Hajj. 2021. AraGPT2: Pre-trained transformer for Arabic language generation. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop*, pages 196–207, Kyiv, Ukraine (Virtual). Association for Computational Linguistics.

Djelloul Bouchiha, Abdelghani Bouziane, Noureddine Doumi, and Benamar Hamzaoui. 2025. Fine-tuning AraGPT2 for hierarchical Arabic text classification. *Science, Engineering and Technology*, 5(1):55–65.

Djelloul Bouchiha, Benamar Hamzaoui, Abdelghani Bouziane, Noureddine Doumi, Farouk Omar Berbouchi, Aymen Abdelghani Kebir, Nihad Mebarki, and Badiâ Achouak Benameur. 2026. Hierarchical Arabic text classification: Deep learning-based approach. *Annals of West University of Timişoara, Mathematics and Computer Science*, 62:1–15.

Fuhan Cai, Duo Liu, Zhongqiang Zhang, Ge Liu, Xiaozhe Yang, and Xiangzhong Fang. 2024. NER-guided comprehensive hierarchy-aware prompt tuning for hierarchical text classification. In *Proceedings of LREC-COLING 2024*, pages 12117–12126.

Huiyao Chen, Yu Zhao, Zulong Chen, Mengjia Wang, Liangyue Li, Meishan Zhang, and Min Zhang. 2024. Retrieval-style in-context learning for few-shot hierarchical text classification. *Transactions of the Association for Computational Linguistics*, 12:1214–1231.

Thomas G. Dietterich. 2000. Ensemble methods in machine learning. In *Multiple Classifier Systems (MCS)*, LNCS 1857, pages 1–15.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2024. Improving factuality and reasoning in language models through multiagent debate. In *Proceedings of ICML*.

Jaco du Toit and Marcel Dunaiski. 2025. Combining language and topic models for hierarchical text classification. *arXiv preprint arXiv:2507.16490*.

Mo El-Haj, Saad Ezzini, Shadi Abudalfa, Salima Lamsiyah, and Mustafa Jarrar. 2026. AraGenre 2026: A hierarchical definition-guided Arabic genre classification shared task. In *Proceedings of the 4th Arabic Natural Language Processing Conference (ArabicNLP 2026)*, Budapest, Hungary. Association for Computational Linguistics.

Gemma Team, Google DeepMind. 2026. Gemma 4 technical report. Technical report, Google DeepMind. ArXiv:2607.02770. Accessed 23 August 2026.

Jonas Golde, Nicolaas Paul Jedema, RaviKiran Krishnan, and Phong Le. 2026. Hierarchical text classification with LLM-refined taxonomies. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (EACL), Long Papers*, pages 214–228.

Google. 2026. Gemini 3.6 flash. Model documentation, https://ai.google.dev/gemini-api/docs/models/gemini-3.6-flash. Accessed 23 August 2026.

Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. 2023. LLM-blender: Ensembling large language models with pairwise ranking and generative fusion. In *Proceedings of ACL*.

Lekang Jiang, Wenjun Sun, and Stephan Goetz. 2025. Reasoning for hierarchical text classification: The case of patents. *arXiv preprint arXiv:2510.07167*.

OpenAI. 2026. GPT-5.6 luna. Model documentation, https://developers.openai.com/api/docs/models/gpt-5.6-luna. Accessed 23 August 2026.

Shuaidong Pan and Di Wu. 2025. Hierarchical text classification with LLMs via BERT-based semantic modeling and consistency regularization. *Preprints.org*.

Qwen Team. 2025. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*.

Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. 2023. Is ChatGPT good at search? investigating large language models as re-ranking agents. In *Proceedings of EMNLP*.

Pat Verga, Sebastian Hofstatter, Sophia Althammer, Yixuan Su, Aleksandra Piktus, Arkady Arkhangorodsky, Minjie Xu, Naomi White, and Patrick Lewis. 2024. Replacing judges with juries: Evaluating LLM generations with a panel of diverse models. *arXiv preprint arXiv:2404.18796*.

Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. 2024. Mixture-of-agents enhances large language model capabilities. *arXiv preprint arXiv:2406.04692*.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In *Proceedings of ICLR*.

Yihong Wang, Zhonglin Jiang, Ningyuan Xi, Yue Zhao, Qingqing Gu, Xiyuan Chen, Hao Wu, Sheng Xu, Hange Zhou, Yong Chen, and Luo Ji. 2025. Making language model a hierarchical classifier. *arXiv preprint arXiv:2507.12930*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Mingxuan Xia, Zhijie Jiang, Haobo Wang, Junbo Zhao, Tianlei Hu, and Gang Chen. 2025. Ensembling prompting strategies for zero-shot hierarchical text classification with large language models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 18189–18208.

Alessandro Zangari, Matteo Marcuzzo, Michele Schiavinato, Lorenzo Giudice, Andrea Albarelli, Andrea Gasparetto, and Matteo Rizzo. 2024. Hierarchical text classification and its foundations: A review of current research. *Electronics*, 13(7):1199.

Biqing Zeng, Yihao Peng, Jichen Yang, Peilin Hong, and Junjie Liang. 2025. CPM-based hierarchical text classification. *Journal of Artificial Intelligence Research*, 82:367–388.

Qian Zhang, Qinliang Su, Wei Zhu, and Yachun Pang. 2025. HierPrompt: Zero-shot hierarchical text classification with LLM-enhanced prototypes. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 3846–3859.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *NeurIPS*.

He Zhu, Chong Zhang, Junjie Huang, Junran Wu, and Ke Xu. 2023. HiTIN: Hierarchy-aware tree isomorphism network for hierarchical text classification. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7809–7821, Toronto, Canada. Association for Computational Linguistics.

A Reproducibility

Official submission. The results we report are those of our final submission, evaluated 1 August 2026, which scored 0.7352 Hierarchical Macro F1 (Specific Macro 0.5725, Weighted 0.6125, Accuracy 0.6189; Broad Macro 0.8979, Weighted 0.9142, Accuracy 0.9138) over 27,972 of 27,972 gold instances with no missing prediction, ranking 1st of the 18 teams in the organisers' final standings. Every hidden-test *metric score* we report for a system configuration comes from an official submitted file rather than a local estimate; the counts and agreement tables elsewhere are local analyses of those files; the full record is in Appendix I. No parameters are updated at any stage; task-specific information enters only through the hand-written discriminators, dev-tuned prompts and leaderboard-guided component selection.

Data: we use only the official AraGenre inputs and definition files, and no external labelled data. The consensus verifiers receive the 74 definitions verbatim. The judge does not: by default it appends a one-line Arabic discriminator to each definition and prepends per-family cue lists,

both author-written from the released definitions and Arabic linguistic knowledge, and both naming genres explicitly (`src/discriminators.py`, `src/judge.py`). The surface rules name a further seven labels. No test label or annotation informs any of it, but the pipeline is not label-agnostic and we do not claim it is.

Provenance of the hand-written material. The 74 discriminators and the per-family cue lists were written by the authors from the released English definitions and Arabic linguistic knowledge, without consulting any labelled data. They were composed during the evaluation phase, after the first scored submission and before the base configuration was scored, and were not revised afterwards; each was therefore fixed before the configuration that used it was submitted. We did not keep per-cue authorship records.

Provenance of the selected constants. None of the following was tuned on held-out gold. The five attractor classes and the seven accepted broad-move directions were chosen by reading our own predictions over the released test inputs (§4). The calibration α=0.5 replaced an initial 0.75 chosen on the released training labels, after the larger value over-flattened the 74-class space (Appendix G). The 2,635-instance verification subset was selected during the evaluation phase as the instances whose broad label we judged least secure. The broad gate is an earlier scored submission of ours reused verbatim. On dev the drain fires on the two most-predicted classes, re-derived there rather than carried over.

Models: the base retriever is Qwen3-Embedding-8B (4-bit NF4 weights with fp16 compute, 4096-dim, cosine over L2-normalised vectors, sequence length capped at 512 tokens); the judge is a Gemma-family model (`gemma-4-31b`) served via vLLM with setwise 0–100 scoring; the consensus verifiers are GPT-5.6 Luna and Gemini-3.6 Flash, each queried with the full-taxonomy prompt.

Settings: the judge decodes at temperature 0 with seed 42; the Gemini verifier is called with `temperature=0` and the GPT verifier with `reasoning_effort=low` and no temperature set. We record the parameters we sent; whether a hosted model honours them is outside our control, and we do not treat `temperature=0` as guaranteeing deterministic output. The retriever caches the top 20 specific genres per text; the judge scores every sibling in the four small families and the top 6 retrieved elsewhere, in a single setwise pass, with marginal correction α=0.5.

Pipeline and code: the code and cached model outputs described in this appendix will be released with the camera-ready version; paths below are relative to the repository root and name what that release contains. Every stage is one file: retrieval (`src/retrieve.py`), the family-restricted judge (`src/judge.py`), the surface rules (`src/rules.py`), the full-context verifiers (`src/verify.py`), the consensus and drain assembly (`src/assemble.py`), and the submission builder (`src/submission.py`), which enforces broad=parent(specific) and validates that all 27,972 IDs are present with exact labels.

Scope of reproducibility. The final submission can be reconstructed exactly from the cached artifacts, but the base pipeline cannot be regenerated end to end. Two inputs to the trap experiment illustrate the gap: the 2,635-instance verification pool, selected during the evaluation phase as the instances whose broad label we judged least secure, and the broad-only prompt behind arm A. Both the pool file and arm A's cached outputs ship, so `src/trap_table.py` rebuilds the table offline, but neither selection can be re-derived from the code.

Two cached inputs. The broad gate the judge restricts to is the scored 0.6812 system, an earlier retrieval-gated system we submitted and scored, reused verbatim rather than regenerated by the released code; and the base predictions additionally carry an Interactive-recall reassignment applied during the evaluation phase by a separate automatic judge pass (`src/interactive_judge.py`), which is included but is not wired into `reproduce.sh`; re-running the judge over the same candidate sets reproduces 93.5% of the base specific labels and 97.1% of the broad labels. Both cached files are included, so the submitted predictions rebuild exactly (27,972/27,972) from cached outputs, but the base stage is a cached input rather than something the repository regenerates end to end.

Caveats: the judge endpoint's serving precision was not recorded (Appendix H), so the base stage is not bit-reproducible even with a GPU; the hosted API models are versioned by release date and are not guaranteed bit-reproducible across dates; the consensus stage incurs API cost proportional to the number of verified instances (on the order of tens of US dollars for this test set). The base pipeline

runs on a single CUDA GPU (embeddings) plus a remote vLLM endpoint (judge).

B Annotation-Convention Divergence Between Splits

Five of the seven residual dev errors are texts that analyse religion from the outside, e.g. تشير بعض الدراسات إلى أن أساليب الوعظ تختلف بين المجتمعات (*tushīru ba'du al-dirāsāti ilā anna asālība al-wa'zi takhtalifu bayna al-mujtama'āt*, "some studies indicate that preaching styles differ between societies"). Our system labels these `Analytical Reports` (Informative); the dev gold label is `Religious Commentary` (Religious).

The released test taxonomy points the other way: `islamic_book_description` ("texts summarising Islamic or religious books") is Informative and our correction stage moves such predictions out of Religious, consistently with the released test definition. The examples suggest different boundary choices across the disjoint dev and test taxonomies, although the differing label inventories prevent a direct annotation-consistency comparison. A system following the released test taxonomy is therefore at a disadvantage on these dev examples, an observed tradeoff rather than a formal bound. It also supports our central claim: the convention is carried by the definitions, which is why supplying the full taxonomy, rather than the broad labels alone, changes model behaviour so sharply.

C Prediction Audit Examples

Three recurring prediction patterns appear in our audit. (i) *Cross-family topic leaks*: a description *of* a religious book (يهدف هذا الكتاب إلى قراءة سيرة الرسول, *yahdifu hādhā al-kitābu ilā qirā'ati sīrati al-rasūl*, "this book aims at a reading of the Prophet's biography", id 10041) is `islamic_book_description` (Informative), which both our base and final systems label that way, consistent with the released definition, but which all three verifiers call Religious under the broad-only prompt; scripture leaks the other way (هؤلاء امراء بني عيسو, *hā'ulā'i umarā'u banī 'Īsū*, "these are the chiefs of the sons of Esau", Genesis 36, id 10798), where the base says `history_encyclopedia` and consensus changes it to `bible`. (ii) *Heavy attractor prediction*: long prose absorbed into `egyptian_song_lyrics`. (iii) *Low cue coverage*: for each of the five song-lyric classes, 87–99% of the texts we assign to it carry no marker from our cue inventory (`src/dialect_markers.py` holds the lists and rebuilds the table). This measures the coverage of our inventory, not an intrinsic ceiling: it shows our system lacks reliable surface cues for these distinctions, not that no written signal exists.

D Negative Results

A *direct* six-way broad classifier scored 0.6206; in our submitted configurations this fell below deriving broad from the specific prediction, though one prompt configuration cannot settle the comparison in general. Scoring all 74 candidates without retrieval pruning fell to 0.6517. For correction, plain self-consistency (0.7134), entropic optimal transport (0.7115), uniform-prior calibration (0.7124) and an unrestricted chain-of-thought pass (0.7131) all regressed below the 0.7139 base, whereas full-context consensus improved it. Cross-lingual rerankers failed (Aarab, 2026), regex rules added nothing the judge lacked, and the off-the-shelf dialect classifiers we tried did not expose the Iraqi/Sudanese distinction that NADI-style country-level tasks define (Abdul-Mageed et al., 2021).

E Full-Context vs. Broad-Only Prompting

Both verifiers were run over the same verification pool ($n = 2{,}635$) once with only the 6 broad definitions and once with all 74 specific definitions. That pool was selected during the evaluation phase as the instances whose broad label we judged least secure, so it is an enriched diagnostic sample: rates measured on it are not representative of all 27,972. Because the full-taxonomy prompt also carries an explicit type-vs-topic instruction, we added a third arm, broad definitions only *plus* that instruction, to separate the two factors. On the 1,403 book-description instances GPT-5.6 calls 394 of them Religious with neither, 233 with the instruction alone and 119 with the full taxonomy; `src/trap_table.py` rebuilds these counts from cached verifier outputs with no API access.

Table 3 splits the same three arms over the whole pool. Two things follow. First, the reduction is not a prior shift: from A to C the verifier calls Religious *more* often on the instances the base already reads as Religious and less often on every

other subset, so the separation between the two widens monotonically. Second, the instruction and the taxonomy are not interchangeable. The instruction pays off only on the book descriptions, where the competing label (`*_book_description`) is the trap's target; on the rest of the pool it overfires, raising Religious calls by a quarter. Only the full-taxonomy arm lowers them on the book descriptions and the rest of the pool while raising them on base-Religious instances, which is what widens the separation. We read this as the instruction supplying a convention the model cannot act on until the taxonomy shows it a label that satisfies the convention. The caveat of Table 4 applies here too: the row labels come from the base system, so these are agreement counts, not accuracy.

Religious calls	A	B	C
book descriptions (n=1,403)	394	233	**119**
rest of pool (n=1,169)	121	152	**61**
base-Religious (n=63)	26	28	**30**
separation (pp)	21	29	**41**

Table 3: The three arms over the full verification pool (GPT-5.6). A: broad definitions only. B: + type-vs-topic instruction. C: + full 74-definition taxonomy. The first two rows should fall and the third should rise; only C does both. Separation is the Religious-call rate on base-Religious instances minus the rate on the other 2,572. Rebuilt by `src/trap_table.py`.

Table 4 reports how often GPT-5.6's broad verdict still agrees with the base system's family, one model across the two original prompts. Broad-only prompting collapses on families whose members are *about* another family's subject matter (Learning, Informative), which is exactly the topic trap; the full taxonomy repairs it. Requiring both verifiers to agree, joint Religious assignments fall from 134 under broad-only prompting to 37 under the full taxonomy, a 72% reduction. That comparison is restricted to the 548 book descriptions for which the second verifier has a call under both prompts, since the deployed full-taxonomy run queries it only on the instances where the first verifier disagrees with the base; on all 1,403, the first verifier alone falls from 394 to 126. That 126 is the deployed all-instance run scored on this subset, not arm C, which is a separate pool-matched run of the same prompt. The two were issued 26 minutes apart on 31 July 2026 with the same model string and the same instances, so the seven-prediction gap is run-to-run variation in the hosted model rather than a difference in configuration; we did not record the exact model build returned by either call. All three broad-only runs are included, and `src/broad_only_check.py` reproduces these counts and the 157 above without API access.

Base family	n	broad-only	full-taxonomy
Informative	1682	49%	**78%**
Learning	305	18%	**74%**
Interactive	153	46%	**72%**
Legal	281	53%	**56%**
Creative	151	50%	48%
Religious	63	41%	**48%**

Table 4: Share of instances where GPT-5.6's broad verdict matches the base family, by prompt context. This measures *agreement with the base system*, not accuracy: the hidden test has no released gold, so we can show that full context changes verifier behaviour and that the change is consistent with the taxonomy's own conventions, but we cannot certify it as an accuracy gain. Dev cannot serve as the gold-anchored counterpart here: it contains no book descriptions and no scripture, so the trap this table measures has no instances there (§6.1).

F Attractor Drain

Relative to the base, the final system changes 459 broad and 2,890 specific labels (10.3% of instances). The drain is deterministic: it fires when the base label is an attractor class and the chain-of-thought pass names a different sibling inside the same broad family (Table 5). No score threshold or entropy criterion is used, and broad is held fixed by construction.

Class	base	moved
`literature_fiction_bk_desc.`	598	208 (35%)
`egyptian_song_lyrics`	979	212 (22%)
`msa_poetry`	608	52 (9%)
`culture_encyclopedia`	455	41 (9%)
`history_encyclopedia`	632	41 (6%)

Table 5: Attractor classes and how many predictions the drain reassigns to within-family siblings when applied to the base predictions alone. In the submitted pipeline consensus is applied first and the drain fires on 383 of these instances.

G Calibration

The correction subtracts a scaled per-genre mean from each candidate's score, $s'(y) = s(y) - \alpha\, \bar{s}(y)$, where $\bar{s}(y)$ is the mean score genre y received over the whole test set and α=0.5. It damps genres the

judge rates generously everywhere, which is what leaves rare siblings seldom predicted. We started from α=0.75, chosen on the released training labels; it over-flattened once the label space grew from 7 to 74 classes. The correction is applied before the argmax and never changes the broad family. Because $\bar{s}(y)$ is a batch statistic, this stage is transductive: a document's label depends on the set it is scored with. Two alternatives we tried instead of it, uniform-prior calibration and entropic optimal transport, both scored below the base (Appendix D).

H Hyperparameters and Prompts

Retriever: Qwen3-Embedding-8B loaded in 4-bit NF4 with double quantisation and fp16 compute (the configuration the submitted run used); only the cosine ranking is consumed, and we did not measure the effect of NF4 quantisation on retrieval rankings, 4096-dim, L2-normalised, `max_seq_length` 512, the top 20 specific genres cached per text; the judge scores every sibling in the four small families (Creative 7, Religious 6, Interactive 4, Legal 3) and the top 6 retrieved in-family candidates in the two large ones.

Judge. `gemma-4-31b` via vLLM, setwise 0–100 scoring in a single pass, temperature 0, seed 42, 3 retries with JSON-regex fallback, marginal correction α=0.5. We did not record the serving precision of the judge endpoint; vLLM's default for this checkpoint is bfloat16. The retriever precision is given above.

Consensus verifiers. GPT-5.6 Luna (`reasoning_effort=low`, $\leq$500 completion tokens) and Gemini-3.6 Flash (called with `temperature=0`, $\leq$400 output tokens), each given the full 74-definition taxonomy (2,884 tokens under `o200k_base`, giving a mean prompt of 3,034 tokens and a maximum of 5,185) in the system prompt, and the text truncated to 1500 characters. The judge prompt asks for a JSON array of integer scores over the candidate definitions; the consensus prompt asks for a single exact `specific_genre` identifier and states that a factual description *of* a book is a `*_book_description`, not the book's subject genre.

Software. `torch`, `transformers`, `sentence-transformers` and `bitsandbytes` for the retriever, vLLM for the local judge, and the `openai` and `google-genai` SDKs for the verifiers; pinned versions accompany the code.

Cost. The $50 total quoted in §5 is the amount actually billed for the 36,013 correction calls at the vendor prices in force in July 2026. We did not log per-request token usage, so we cannot decompose it into input and output totals or restate it as a rate.

Throughput. A consensus call takes 2.0–2.8 s. At the released default of 20 concurrent workers the 8,041-instance Gemini pass took 19 min of wall clock; the full GPT pass over 27,972 instances was run at higher concurrency.

I Official Submission Record

Every hidden-test metric score we report for a system configuration comes from an official submitted file (Table 6), so the progression is traceable to scored submissions rather than to local approximations. Counts such as the arm totals and the agreement tables are local analyses of those files, not scored quantities.

System	Hier. F1
genre-general judge, no retrieval	0.6067
+ embedding retrieval gate	0.6812
+ family-restricted judging	0.6882
direct six-way broad judge	0.6206
unpruned 74-way scoring	0.6517
+ surface rules only	0.6881
rule variant	0.6891
prompt-ensemble blend	0.6883
Interactive-subtype judging	0.6942
hybrid judge variant	0.7128
base system	0.7139
uniform-prior calibration	0.7124
self-consistency (K=3)	0.7134
entropic optimal transport	0.7115
chain-of-thought applied wholesale	0.7131
+ agreement-gated corr. (subset)	0.7224
+ cross-family moves, all inst.	0.7256
+ within-family refs, drain	**0.7352**

Table 6: Official submissions behind Table 1 and Appendix D. The final system produced 27,972 predictions over 27,972 gold instances, none missing. We used 19 of our 26 permitted submissions; the 18 scored on the test phase are all listed here. Submissions were made sequentially over four days, so the progression is a record of scored configurations rather than a single controlled sweep.

The scale of a stage gain. Every specific class carries the same $1/74$ weight in Specific Macro F1 regardless of how often it is predicted, so moving

one class's F1 from 0 to 1 shifts Specific Macro F1 by 0.0135 and Hierarchical Macro F1 by 0.0068. Measured against that unit the individual stages in Table 1 are small: the correction on the subset is worth about 1.25 class-level effects, extending the cross-family moves about 0.47, and the within-family refinements with the drain about 1.41. The full base-to-final gain is about 3.1 and the margin over the second-placed system about 2.7. These are coarse yardsticks, not significance tests.

Two readings this record supports. The 0.6881 row isolates the surface rules on top of the 0.6882 row: they rewrite 32 labels (23 `quran`, 9 `hadith`) and leave the score at 0.6881 against 0.6882. The rules had an approximately score-neutral effect in this configuration, so the 0.6882→0.7139 gain belongs to the judge and calibration changes, not to them. They were kept for a different job: in the final pipeline they fire on 1,087 instances and agree with the base label on every one, so their whole net effect is to restore 75 labels that consensus or the drain had moved away (56 `youtube_comments`, 19 `quran`). They guard the correction stages rather than adding score. The chain-of-thought row applies that pass to every instance and scores 0.7131, below the 0.7139 base; the same pass restricted to attractor classes is what the drain uses. Chain-of-thought helps here only where it is aimed.

Gated broad moves. Of the 3,300 instances where both verifiers agree against the base, 2,122 stay inside the base family and are applied as refinements. Of the 1,178 that would cross families, 460 fall in an author-screened direction and are applied: Informative→Interactive 195, Legal→Informative 80, Creative→Religious 58, Informative→Religious 58, Creative→Informative 44, Interactive→Religious 20, Legal→Interactive 5; the final file differs from the base on 459 broad labels, one fewer, because the closing surface-rule pass restores one. The remaining 718 are rejected, the largest groups being Informative→Learning (219), Creative→Interactive (122) and Learning→Informative (99).

Preserved rule defects. Three regexes in `src/rules.py` do not match their intent: the Qur'anic class also admits U+0671, the opinion class lets a bare variation selector match, and one character class contains a literal `|`. They ran as written when the submission was produced, so we keep them verbatim and document them in the code; correcting all three changes 22 of the 27,972 final predictions.

Orthographic signals. Share of each group carrying heavy diacritisation (a mark on over 10% of its Arabic characters) or at least one emoji, over the final predictions: `quran` 77%/0% (n=691), poetry 43%/0% (n=1,606), `hadith` 1%/1% (n=647), book descriptions 1%/0% (n=3,916), the whole Interactive family 0%/28% (n=1,978; `twitter_posts` 57%, `youtube_comments` 30%, `app_reviews` 11%, `book_reviews` 0%). Both are properties of the text, so no gold is needed; `src/orthographic_signals.py` reproduces the table.